

DATABASE MANAGEMENT SYSTEMS

Technical Presentation

ER / EER Modeling • DBMS Architecture • Data Independence • Entity Design

Course Code: CSA05

Assessment Tool: AT-3 – Technical Presentation (Low)

Course Outcome: CO1 – ER/EER Diagrams & Relational Schema Design

ER Diagram: Online Library System

Understanding entities, attributes and relationships before drawing the model

E

What is an ER Diagram?

A visual blueprint that models real-world data as entities, their attributes, and the relationships that connect them.



Why the Library System?

A library involves clear, relatable entities — Book, Member, Author, Staff — making it ideal to demonstrate ER concepts.



Design Goal

Capture every rule of the library (who can borrow what, and how many copies exist) accurately in the diagram.



Outcome

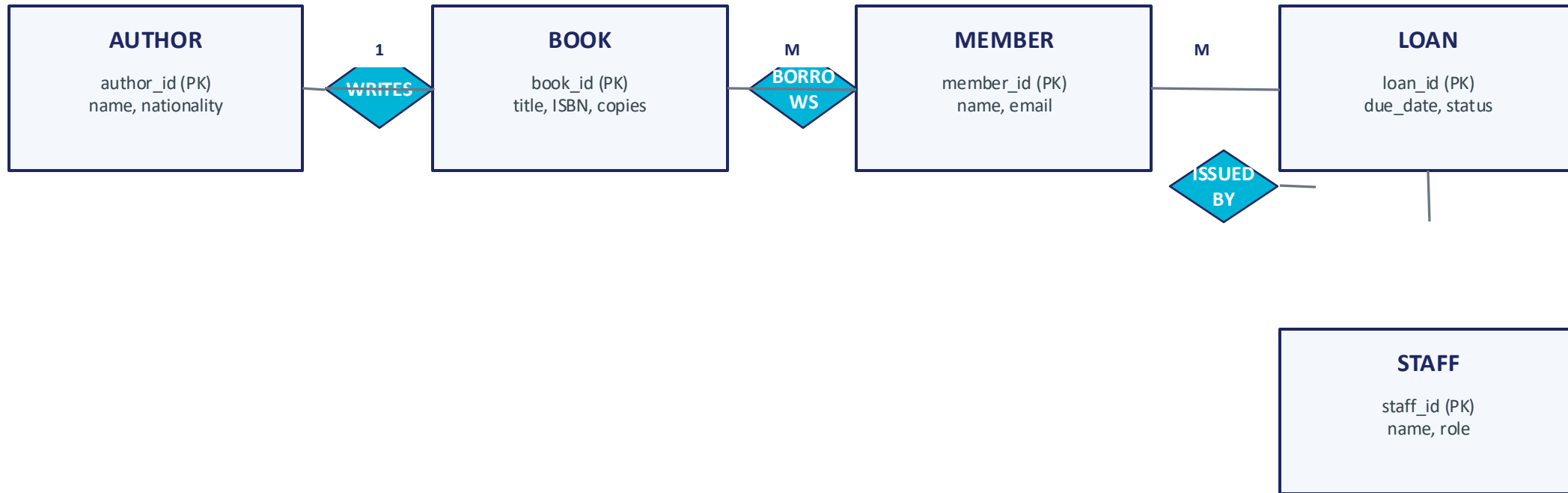
A schema-ready diagram that can be converted directly into relational database tables.



ER diagrams remove ambiguity — every business rule of the library is expressed visually before coding begins.

Entity-Relationship Model

Core entities and relationships of the Online Library System



Legend: 1 = one | M = many | ◇ relationship | □ entity | underline = primary key



One Author WRITES many Books; one Member BORROWS many Books via Loan; each Loan is ISSUED BY one Staff.

Attributes & Cardinality Rules

Detailing keys, attribute types and participation constraints

Entity	Key Attribute	Other Attributes	Cardinality
Book	book_id (PK)	title, ISBN, copies_available	1 Author : M Books
Member	member_id (PK)	name, email, membership_date	1 Member : M Loans
Author	author_id (PK)	name, nationality	1 Author : M Books
Staff	staff_id (PK)	name, role, shift	1 Staff : M Loans issued
Loan	loan_id (PK)	issue_date, due_date, status	M Loans : 1 Book, 1 Member



Primary keys guarantee uniqueness; cardinality notation defines exactly how records relate before the schema is built.

Key Takeaways

1

The Library ER model captures 5 entities linked by 3 meaningful relationships.

2

Cardinality (1:M) rules prevent data anomalies such as a loan without a valid book or member.

3

Every entity's primary key becomes a foreign key when the diagram is mapped to relational tables.

4

A well-designed ER diagram is the single most important step before physical database creation.

EER Model: Generalization & Specialization

Extending the ER model with class hierarchies — University case study



Generalization

A bottom-up process: combining common attributes of lower-level entities into a single higher-level superclass.



Specialization

A top-down process: splitting a superclass into specialized subclasses that share its attributes plus their own.



Inheritance

Every subclass automatically inherits all attributes and relationships of its superclass (IS-A relationship).



Constraints

Disjoint / overlapping and total / partial constraints define how subclasses relate to the superclass.

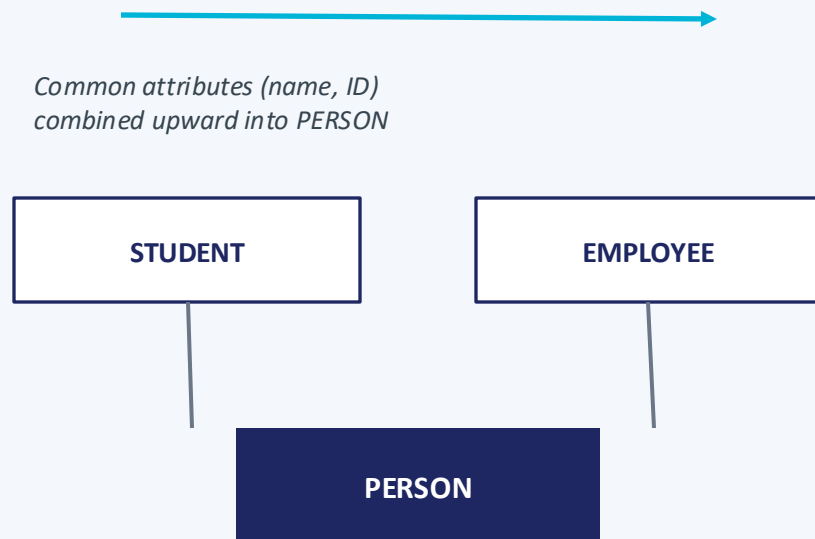


EER modeling lets a University database represent Students and Employees as specialized types of one Person superclass.

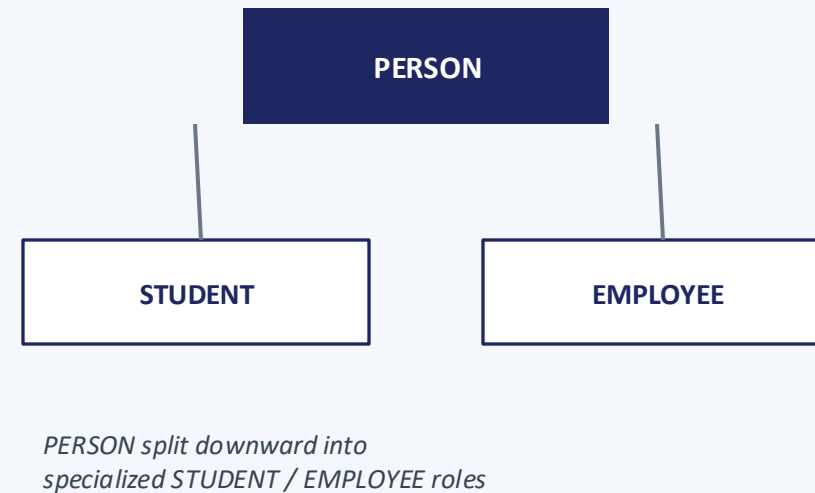
Generalization vs Specialization

Two directions of building the same class hierarchy

GENERALIZATION (Bottom-Up)

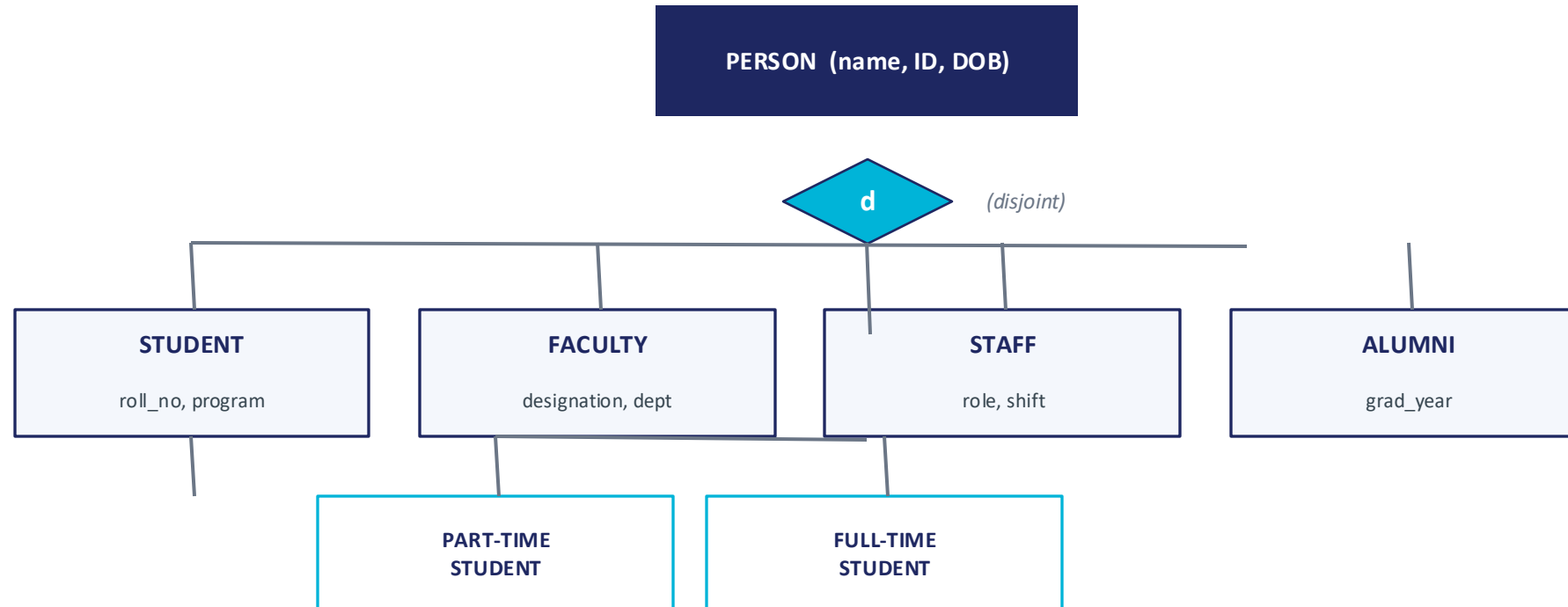


SPECIALIZATION (Top-Down)



University Case Study Hierarchy

Applying disjoint & overlapping specialization to real University roles



A Faculty member can never also be a Student (disjoint, 'd'), while Student itself further specializes into Part-time / Full-time.

Why EER Modeling Matters

Benefits of generalization & specialization in real database design



Reduces Redundancy

Shared attributes like name and ID live once in PERSON instead of being repeated in every subclass.



Reflects Real Rules

Disjoint/overlapping constraints mirror real institutional policy — e.g., a person cannot be Faculty and Student at once.



Scales Cleanly

New roles (e.g., Visiting Scholar) can be added as a new subclass without redesigning the whole schema.



Maps to Tables

Each subclass converts to its own relational table linked to PERSON by a shared primary key (ID).



EER modeling is the bridge between simple ER diagrams and complex, real-world hierarchical data like University roles.

Data Independence in DBMS

Shielding applications from the impact of database changes

What is Data Independence?

□ The capacity to change a schema at one level of the database without needing to change the schema at the next higher level.

Logical Data Independence

L Ability to change the conceptual schema (e.g., add a new entity) without altering external schemas or application programs.

Physical Data Independence

P Ability to change the internal (storage) schema — indexes, file structures — without altering the conceptual schema.

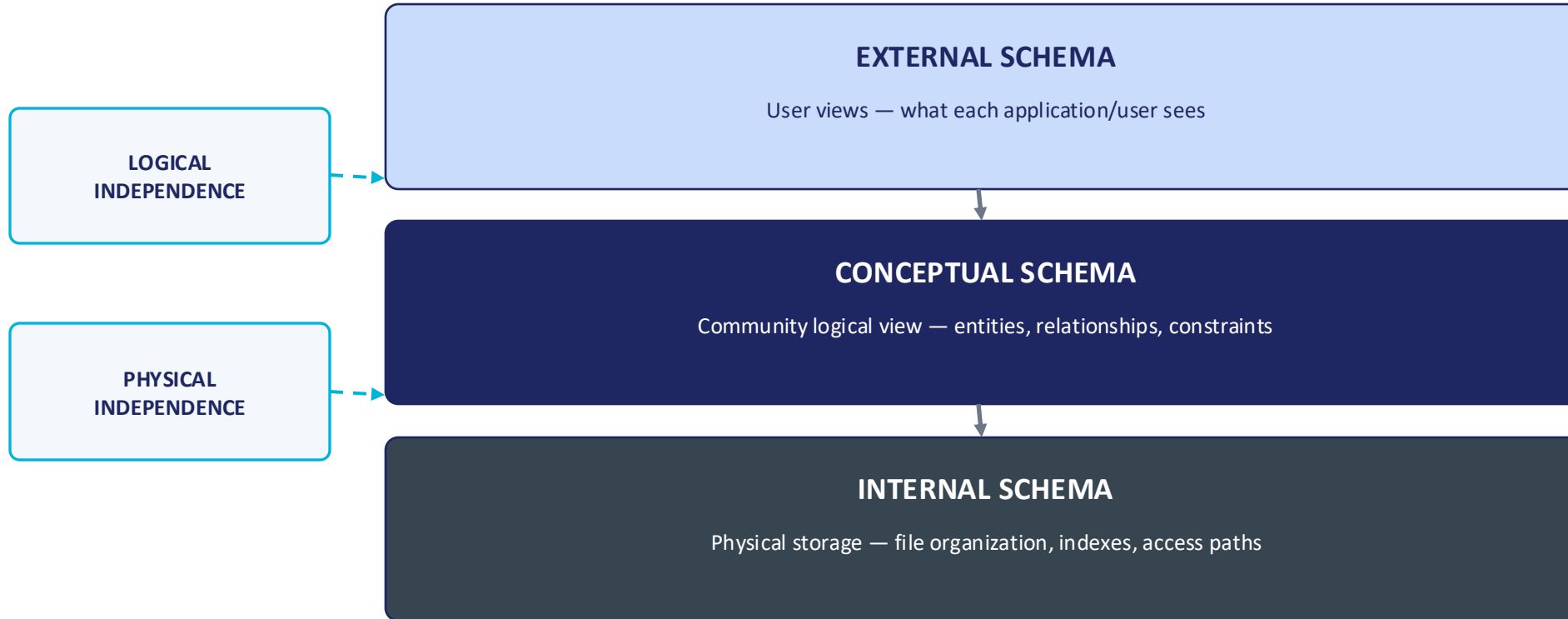
Why It Matters

★ It minimizes maintenance cost and lets databases evolve safely alongside growing applications.

★ **Data independence is the foundation of the ANSI/SPARC three-schema architecture used in every modern DBMS.**

Three-Schema Architecture

Where logical and physical independence operate



Logical independence isolates External from Conceptual; physical independence isolates Conceptual from Internal.

Logical vs Physical Independence

Side-by-side comparison of scope, triggers and impact

Aspect	Logical Data Independence	Physical Data Independence
Schema Level Affected	Conceptual schema	Internal (storage) schema
Typical Change	Add/remove an entity or attribute	Change indexing or file structure
Who is Shielded	External schema & application programs	Conceptual schema
Difficulty to Achieve	Harder — many dependent views	Easier — fewer dependencies
Example	Adding a Membership_Type to Members	Switching from B-tree to hash index



Logical independence protects against conceptual-level changes; physical independence protects against storage-level changes.

Significance in DBMS Architecture

1

Reduces application maintenance — programs need not be rewritten when the schema evolves.

2

Enables performance tuning (new indexes, partitioning) without breaking existing queries.

3

Supports multiple external views over one shared conceptual schema, serving many user groups.

4

Forms the theoretical backbone of the ANSI/SPARC architecture adopted by all major DBMS products.

Software Components of a DBMS

Overview of the core engine that powers every database system

Query Processor

Parses, optimizes, and prepares an execution plan for every SQL statement submitted to the DBMS.

Storage Manager

Manages how data is physically stored, retrieved and buffered on disk — files, indexes, and buffers.

Transaction Manager

Ensures ACID properties — Atomicity, Consistency, Isolation, Durability — for every transaction.

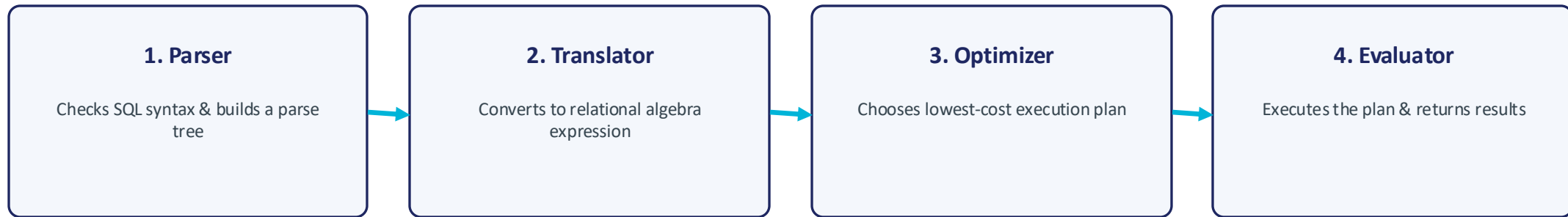
How They Cooperate

A single query flows through all three components before a result is returned to the user.

These three components together form the 'engine room' of the DBMS, invisible to the end user but critical to every operation.

Query Processor — Breakdown

From SQL text to an executable, optimized plan



Query optimization is the most performance-critical stage — it decides which indexes and join strategies to use.

Input: `SELECT * FROM Book WHERE genre = 'Fiction';`

Output: Optimized plan → index scan on genre → fetch matching rows

Storage & Transaction Manager

The layer responsible for safe, durable, and consistent data

STORAGE MANAGER

1

Authorization & Integrity Manager

Checks access rights and constraints

2

File Manager

Manages allocation of disk space & structures

3

Buffer Manager

Moves data between disk and main memory

TRANSACTION MANAGER

A

Atomicity:

All-or-nothing execution of operations

C

Consistency:

Database moves between valid states only

I

Isolation:

Concurrent transactions do not interfere

D

Durability:

Committed changes survive system failure



Storage Manager handles WHERE data lives; Transaction Manager guarantees HOW SAFELY changes to that data are made (ACID).

How the Components Interact

Full data-flow: from a user query to a committed transaction



Request flows left → right; commit acknowledgment flows right → left



Every read/write request is coordinated across all three engine components to guarantee correctness and durability.

- The Query Processor decides WHAT to do.
- The Transaction Manager decides HOW SAFELY to do it.
- The Storage Manager decides WHERE the data physically lives.

Strong vs Weak Entities in ER Modeling

Distinguishing independent entities from dependent ones

S

Strong Entity

Has its own primary key and can be uniquely identified independent of any other entity.

W

Weak Entity

Has no primary key of its own; it is identified only through a partial key combined with a related strong (owner) entity.

→

Identifying Relationship

The relationship that connects a weak entity to its owner strong entity, shown with a double diamond.

⌘

Real-World Analogy

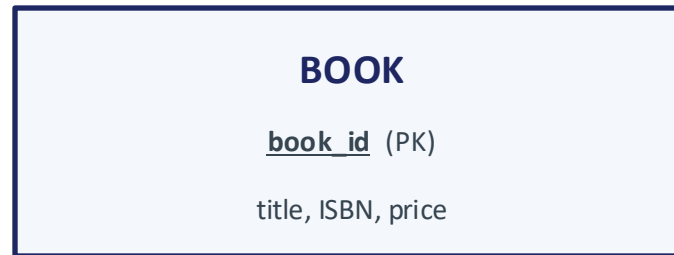
An Order (strong) owns Order_Items (weak) — an item has no meaning without its parent order.

★

Recognizing strong vs weak entities correctly determines how foreign keys and composite keys are designed later.

Strong Entity — Example: BOOK

A single, solid-bordered rectangle with an underlined primary key



- ✓ Drawn as a single-outline rectangle in ER notation.
- ✓ `book_id` alone is enough to uniquely identify every row.
- ✓ Can exist in the database independently of any other entity.
- ✓ Participates in relationships as the 'owner' side when needed.

Weak Entity — Example: BOOK_COPY

A double-outlined rectangle tied to BOOK by an identifying relationship



Full identifying key of BOOK_COPY = book_id (from BOOK) + copy_no



BOOK_COPY cannot exist without a BOOK; deleting a Book cascades to delete all its Book_Copy records.

Strong vs Weak Entity — Comparison

Summary table with real-world Library examples

Feature	Strong Entity	Weak Entity
Primary Key	Has its own primary key	No independent primary key
ER Notation	Single-outline rectangle	Double-outline rectangle
Relationship	Normal diamond	Double (identifying) diamond
Existence	Exists independently	Depends on its owner entity
Library Example	BOOK, MEMBER	BOOK_COPY, LOAN_FINE



Correctly identifying strong vs weak entities ensures accurate composite keys and cascading rules in the final schema.